

## AR09: Buzzer

Prof. Irineu Lopes Palhares Junior

FCT/UNESP,  
[irineu.palhares@unesp.br](mailto:irineu.palhares@unesp.br)



# Sumário

- 1 Buzzer
- 2 Circuito
- 3 Notas musicais
- 4 Código
- 5 Desafio de aplicação
- 6 Referências

# Buzzer

Um *buzzer* (ver Fig. 1) é um componente transdutor que converte sinais elétricos em ondas sonoras para criar alertas, bipes ou melodias. Este dispositivo é utilizado para sinalização sonora, sendo aplicado em computadores, despertadores, carros, entre outros. O buzzer é composto por duas camadas de metal, uma terceira camada de cristal piezelétrico, envolvidas em um invólucro de plástico, e dois terminais para ligação elétrica.



## Uso e tipos de *buzzer*

O *buzzer* é um dispositivo piezoelétrico de sinalização. Você pode usá-lo de diversas maneiras no seu projeto, por exemplo, um clique, um beep ou som curto podem indicar, por exemplo, que um botão foi pressionado, ou que um sensor foi ativado.

Existem dois tipos principais de buzzer:

- **Ativo:** Emite som contínuo em uma única frequência quando recebe alimentação (liga/desliga como um LED).
- **Passivo:** Precisa de um sinal de ondas quadradas (frequências variadas) para gerar diferentes tons e notas musicais.

## Circuito para o módulo *buzzer*

A Fig. 2 apresenta a montagem do circuito para o modulo *buzzer* ativo.

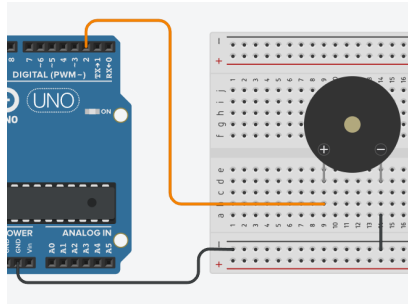


Figura 2: Montagem do circuito de um *buzzer*.

■ FÁ-349 HZ

■ DÓ-523 HZ

## Curiosidades sobre as frequências musicais

A frequência é a característica através da qual o ouvido distingue se um som é agudo ou grave. Esta característica está relacionada com a quantidade de ciclos (vibrações) de uma onda sonora em um período de um segundo, e é expressa em Hertz (Hz).

O espectro de frequências que o ouvido humano pode entender engloba sons entre 20 Hz e 20.000 Hz ou 20 kHz (1 kHz = 1 Kiloherتز ou 1000 Hz).

Aplicando um sinal elétrico em uma determinada frequência, o *buzzer* produz uma nota musical. As notas variam conforme a frequência utilizada.

## Definindo notas no Arduino

- `const float c = 261.63; //Do (Oitava 0)`
- `const float d = 293.66; //Re (Oitava 0)`
- `const float e = 329.63; //Mi (Oitava 0)`
- `const float f = 349.23; //Fa (Oitava 0)`
- `const float g = 392.00; //Sol (Oitava 0)`
- `const float gS = 415.30; //Sol# (Oitava 0)`
- `const float a = 440.00; //La (Oitava 0)`
- `const float b = 466.16; //La# (Oitava 0)`

## Notas oitavadas

- `const float cH = 523.25; //Do (Oitava 1)`
- `const float cSH = 554.37; //Do# (Oitava 1)`
- `const float dH = 587.33; //Re (Oitava 1)`
- `const float dSH = 622.25; //Re# (Oitava 1)`
- `const float eH = 659.26; //Mi (Oitava 1)`
- `const float fH = 698.46; //Fa (Oitava 1)`
- `const float fSH = 739.99; //Fa# (Oitava 1)`
- `const float gH = 783.99; //Sol (Oitava 1)`
- `const float gSH = 830.61; //Sol# (Oitava 1)`
- `const float aH = 880.00; //La (Oitava 1)`

## Código simples com *buzzer*

A Figura 3 apresenta um código simples com o módulo *buzzer*, onde a nota lá é acionada por meio da função *tone()*.

```
int pin=2;

const float c = 261.63; //Do (Oitava 0)
const float d = 293.66; //Re (Oitava 0)
const float e = 329.63; //Mi (Oitava 0)
const float f = 349.23; //Fa (Oitava 0)
const float g = 392.00; //Sol (Oitava 0)
const float gS = 415.30; //Sol\# (Oitava 0)
const float a = 440.00; //La (Oitava 0)
const float b = 466.16; //La\# (Oitava 0)
const float cH = 523.25; //Do (Oitava 1)
const float cSH = 554.37; //Do\# (Oitava 1)
const float dH = 587.33; //Re (Oitava 1)
const float dSH = 622.25; //Re\# (Oitava 1)
const float eH = 659.26; //Mi (Oitava 1)
const float fH = 698.46; //Fa (Oitava 1)
const float fSH = 739.99; //Fa\# (Oitava 1)
const float gH = 783.99; //Sol (Oitava 1)
const float gSH = 830.61; //Sol\# (Oitava 1)
const float aH = 880.00; //La (Oitava 1)

void setup(){
  pinMode(pin, OUTPUT);
}

void loop(){
  //Nota lá
  tone(a, 440);
```

## Detalhes sobre a função *tone()*

A função *tone()* tem como parametros de entrada o pino, frequência e duração da nota, repectivamente, isto é, *tone(pino, frequencia, duracao)*. Quando não especificamos a duração da nota, ela será executada indefinidamente até que se use a função *noTone()* ou outra nota seja executada na sequência.

## Acionar o *buzzer*

Vamos acionar uma música e uma sirene a partir do *buzzer*. Para isso, iremos criar funções com o Arduino.

```
void loop(){
  //Musica:
  Inicio();
  Var1();
  Inicio();
  Var2();
  //Sirene
  Sirene();
}
void Sirene(){
  for (int i=0; i<180; i++){
    float x = (i/180.0)*PI;
    tone(pin,440.0 + sin(x)*100.0);
    delay(5);
  }
}
void Inicio(){
  tone(pin,a,250);
  delay(500);
  tone(pin,a, 250);
  delay(500);
  tone(pin,a, 250);
  delay(500);
  tone(pin,a, 250);
  delay(500);
}
void Var1(){
  tone(pin,cH, 250);
  delay(300);
  tone(pin,dH, 250);
  delay(300);
}
void Var2(){
  tone(pin,g, 250);
  delay(300);
  tone(pin,gS, 250);
  delay(300);
  tone(pin,a, 250);
  delay(500);
}
```

## Desafio de aplicação

Modificar o acionamento da música e sirene a partir do botão de pressão.



## Referências

- 1 <https://www.blogdarobotica.com/2020/10/05/ utilizando-o-buzzer-ativo-no-arduino/>